Minimum Operations to Sort an Array

Problem Statement

You are given an integer array nums of length n, where nums is a permutation of the integers from 0 to n - 1.

You may perform only the following operations:
1. Reverse the entire array.
2. Rotate Left by One: Move the first element to the end of the array, and shift all other elements left by one position.

Return an integer denoting the minimum number of operations required to sort the array in increasing order. If it is not possible to sort the array using only the given operations, return -1.

Example 1:
Input: nums = [0,2,1]
Output: 2
Explanation: Rotate left by one: [2,1,0]. Reverse the array: [0,1,2]. The array becomes sorted in 2 operations, which is minimal.

Example 2:
Input: nums = [1,0,2]
Output: 2
Explanation: Reverse the array: [2,0,1]. Rotate left by one: [0,1,2]. The array becomes sorted in 2 operations, which is minimal.

Example 3:
Input: nums = [2,0,1,3]
Output: -1
Explanation: It is impossible to reach [0,1,2,3] using only the given operations.

Constraints:
1 <= n == nums.length <= 10^5
0 <= nums[i] <= n - 1
nums is a permutation of integers from 0 to n - 1.

Approach

Use the structure of the two allowed operations.

Let L be Rotate Left by One and R be Reverse.

Any sequence of operations can be reduced to one of these forms:
- L^m: rotate left m times
- L^m R: reverse the array, then rotate left m times

This is because two reverses cancel, and reversing after a rotation is equivalent to rotating in the opposite

direction before reversing.

Therefore, nums can be sorted only if it is either:
1. A rotation of the sorted array [0, 1, ..., n - 1], or
2. A reflected rotation of the sorted array.

Case 1: Rotation

If nums[i] = (i + k) mod n for a fixed k, then nums is a rotation of the sorted array.

To sort it:
- Rotate left n - k times, costing n - k operations; or
- Reverse, rotate left k times, then reverse, costing k + 2 operations.

The answer is min(n - k, k + 2), except when k = 0, where the array is already sorted and the answer is 0.

Case 2: Reflected rotation

If nums[i] = (c - i) mod n for a fixed c, then reversing the array produces a rotation of the sorted array.

After one reverse, rotate left n - 1 - c times to sort the array.

The cost is:
1 + min(n - 1 - c, n - (n - 1 - c))

If neither condition holds, return -1.

Example: nums = [1, 2, 0]

Here nums[i] = (i + 1) mod 3, so it is a rotation with k = 1.

- Rotate left twice: [1,2,0] -> [2,0,1] -> [0,1,2], cost 2.
- Reverse, rotate left once, reverse: cost 3.

Therefore, the minimum number of operations is 2.

Java Solution

```
class Solution {
    public int minOperations(int[] nums) {
        int n = nums.length;

        int rotationOffset = -1;
        boolean isRotation = true;

        for (int index = 0; index < n; index++) {
            int expectedOffset = (nums[index] - index + n) % n;

            if (rotationOffset == -1) {
                rotationOffset = expectedOffset;
```

```
            } else if (rotationOffset != expectedOffset) {
                isRotation = false;
                break;
            }
        }

        if (isRotation) {
            if (rotationOffset == 0) {
                return 0;
            }

            int rotateLeftCount = n - rotationOffset;
            int reverseRotateReverseCount = rotationOffset + 2;

            return Math.min(
                    rotateLeftCount,
                    reverseRotateReverseCount);
        }

        int reflectionOffset = -1;
        boolean isReflectedRotation = true;

        for (int index = 0; index < n; index++) {
            int expectedOffset = (nums[index] + index) % n;

            if (reflectionOffset == -1) {
                reflectionOffset = expectedOffset;
            } else if (reflectionOffset != expectedOffset) {
                isReflectedRotation = false;
                break;
            }
        }

        if (isReflectedRotation) {
            int rotationsAfterReverse =
                    (n - 1 - reflectionOffset + n) % n;

            return 1 + Math.min(
                    rotationsAfterReverse,
                    n - rotationsAfterReverse);
        }

        return -1;
    }
}
```

Complexity

Time Complexity: O(n), because each index is checked a constant number of times.

Space Complexity: O(1), since only a few variables are used.